

Guion de sustentación

Parcial 1 — Escenario 5.2 Centro de datos: flota heterogénea de 10 dispositivos sobre Azure IoT Central

Estudiante	Marcos Valera Daza
Aplicación	dcandes1unab · https://dcandes1unab.azureiotcentral.com
Repositorio	https://github.com/MarcozVD/parcial1-iot-central-unab
Duración de la demo	12–15 min + preguntas

Cómo usar este guion: sigue el orden de la sección 1 para la demo en vivo; la sección 3 tiene las respuestas listas para las preguntas del ingeniero y la 4 indica qué enseñar según cada indicador de evaluación.

Para: sustentación presencial ante el ingeniero a cargo. **Duración objetivo:** 12–15 min de demo + preguntas.

Regla de oro: cada afirmación que hagas debe poder señalar una pantalla o un archivo del repositorio. Si algo no arranca en vivo, se muestra la evidencia histórica y se explica el porqué (el pliego lo permite explícitamente para los orígenes que dependen de cuota o de red).

0. Checklist previo (30 minutos antes)

#	Qué	Comando / acción	Cómo sabes que está bien
1	App de IoT Central viva	abrir https://dcandes1unab.azureiotcentral.com	aparece *DC-ANDES-1 · AndesCloud UNAB*
2	Entorno Python	<code>`.venv\Scripts\python.exe -c "import azure.iot.device, paho; print('ok')"`</code>	imprime `ok`
3	Credenciales presentes	<code>`dir .secrets\env`</code>	aparecen los `DC-*.env` (no están en git)
4	Payload de Wokwi regenerado	<code>`python tools/wokwi_payload.py rackb`</code>	escribe <code>`tools/js/wokwi_rackb_payload.js`</code>
5	Red	<code>`ping 8.8.8.8`</code>	responde (los nodos necesitan salida a Internet)
6	Panel listo	abrir https://dcandes1unab.azureiotcentral.com/dashboards	se ve *Cuarto de Control DC-ANDES-1*

Nota importante: los payloads de Wokwi no están en el repositorio (llevan la clave del dispositivo incrustada). Se regeneran con el paso 4 antes de cada sesión; si falta, el paso 4 lo crea.

1. Guion de la demo (orden recomendado)

Minuto 0–2 · Escenario, Digital Twin y catálogo

Di:

"El escenario es el 5.2: un centro de datos urbano, DC-ANDES-1, operado por AndesCloud S.A.S. Tiene tres racks, contención de pasillo frío, climatización por free-cooling, sala de energía con PDU por fila y control de acceso. Sobre eso montamos una flota de diez dispositivos con diez orígenes de envío distintos: no es un simulador repetido diez veces."

Muestra: la plantilla publicada (**Plantillas de dispositivo → Nodo DC-ANDES-1**), sus capas (27 telemetrías, 11 propiedades, 4 comandos) y las dos vistas (Overview, About).

Frase para el catálogo:

"Cada fila del catálogo cambia el código, el feed o el transporte. La tabla de trazabilidad está en `docs/01-catalogo-dispositivos.md`: por eso cada uno cuenta como origen distinto."

Minuto 2–6 · Los dos códigos en vivo, en dos equipos

Equipo 1 — Python en el portátil (nodo del Rack C):

```
cd C:\Users\mvale\Documents\Parcial1_IoT_Central
.\venv\Scripts\python.exe dispositivos\python\dc_sdk_mqtt.py
```

Qué se ve en la consola: [DPS] asignado a iotc-... → [HUB] CONNECT rc=0 → [TX] {...} cada 30 s.

Equipo 2 — ESP32 virtual en Wokwi (nodo del Rack B), en el navegador:

```
:: 1) abrir la sesion del navegador (si no esta abierta; el perfil guarda la sesion de Azure)
playwright-cli -s=iot --browser msedge --persistent --profile
"C:/Users/mvale/AppData/Local/Temp/edgeprof" open "https://wokwi.com/projects/new/esp32"

:: 2) generar e inyectar el sketch + diagrama (el payload regenerado lleva el secrets.h dentro)
python tools\wokwi_payload.py rackb
python tools\pw.py js tools/js/wokwi_rackb_payload.js --tab 0
python tools\pw.py js tools/js/wokwi_tabs.js --tab 0
python tools\pw.py js tools/js/wokwi_rackb_payload.js --tab 0

:: 3) instalar librerias y arrancar
python tools\pw.py js tools/js/wokwi_libs_agua.js --tab 0
python tools\pw.py js tools/js/wokwi_reiniciar3.js --tab 0
```

Qué se ve en el monitor serie: [WIFI] conectado → [NTP] → [DPS] resp=200 status=assigned → [MQTT] conectado al hub (Rack B) → [TX] {...} (ok) cada 15 s.

Si el proyecto de Wokwi tarda en compilar (cola del plan gratuito), espera o abre la pestaña de nuevo; en el guion de contingencia está el plan B.

Y en el portal (lo que mira el ingeniero): *Dispositivos → DC-RACKC-03* y *DC-RACKB-02* en estado **Conectado**, con "Última recepción de datos" avanzando.

Di:

"Son dos equipos y dos códigos distintos: uno es Python con el SDK sobre MQTT/TLS en el puerto 8883, el otro es un ESP32 con firmware Arduino y PubSubClient que se autentica por DPS con SAS simétrico. Los dos publican en vivo sobre la misma plantilla."

Minuto 6–9 · Desconexión controlada y reconexión

Elige **uno** de los dos métodos (el segundo es el más visual):

Método A — matar el nodo Python y relanzarlo:

- Cierra la ventana del nodo del Rack C (o Ctrl+C).
- En el portal, *DC-RACKC-03* pasa a **Desconectado** en ~45–60 s.
- Vuelve a lanzar el mismo comando: el estado regresa a **Conectado** y la serie continúa.
- Señala el hueco en el gráfico del Data Explorer.

Método B — bloquear el dispositivo desde el portal (recomendado):

- *Dispositivos → DC-RACKB-02 → Administrar dispositivo → Bloquear* y confirmar.
- A los ~45 s el dispositivo queda **Desconectado** y el ESP32 empieza a reintentar (se ve en el monitor serie: [MQTT] sesión caída o sin conectar: reconectando).
- *Desbloquear*: vuelve a **Conectado** y siguen los [TX].

Di (esto es lo que sube nota, porque es la desconexión real que pide el pliego):

"En la ventana de cuatro fechas tuvimos cuatro incidentes reales, no simulados: la migración de la flota a un servidor Ubuntu a mitad de ventana, una doble publicación de los mismos dispositivos, el rechazo de SAS del ESP32 por deriva de reloj y la saturación de la cola de compilación de Wokwi. Los cuatro están en la sección 6.1 del informe con su diagnóstico y su resolución."

Minuto 9–12 · Cuarto de control y gráfico de los 4 días

- Abre el panel **Cuarto de Control DC-ANDES-1** (<https://dcandes1unab.azureiotcentral.com/dashboards>): logo del escenario, KPIs, ocho gráficos y el bloque de alertas.
- Explora la flota: *Dispositivos* → 10 dispositivos, y el detalle de uno con pestaña

Datos sin procesar.

- Gráfico de los 4 días no continuos: *Analizar → Explorador de datos*:
- Grupo de dispositivos: **Nodo DC-ANDES-1 - All devices**
- Telemetría: Temperatura intake (rack), Temperatura pasillo frio, Temperatura exterior, PM2.5 en sala, Humedad de piso tecnico, Temperatura de techo, Potencia de fila, Eventos de acceso acumulados
- **Agrupar por: Id. de dispositivo**
- Rango: **24/09/2026 00:00 → 27/09/2026 23:59** (personalizado, no uno de los rápidos)
- Analizar y luego **Guardar** para dejar la consulta en el portal.
- Enseña los huecos entre fechas: son las cuatro fechas no continuas que pide el taller.

Di:

"Cada intervalo de la flota es distinto —15, 30, 45, 60, 120 y 900 segundos—, así que las series tienen densidades muy diferentes: el recuento por variable es la evidencia directa de la asincronía."

Minuto 12–15 · Comparativa y cierre

Muestra la tabla de las cuatro fechas (**24-sep 6 h 01 · 25-sep 8 h 22 · 26-sep 18 h 09 · 27-sep 4 h 32 = 35 h 38 min**) y un par de lecturas operativas:

- **Humo 0,10 %obs/m el 26-sep**: el único valor de la ventana que supera el umbral de la regla (0,08).
- **Corriente 30,82 A el 26-sep**: el punto más cercano a la protección de 32 A de la PDU.
- **6 049 eventos de acceso el 26-sep** frente a 1 393 el 24-sep: jornada de mantenimiento, y explica que ese día la presión diferencial fuera la mínima (más puertas abiertas).

2. Por qué cada origen es distinto (tabla para responder de memoria)

#	Origen	Qué cambia frente a los demás
01	Simulador nativo de la plantilla	no hay código propio: lo genera la plataforma
02	ESP32 de Wokwi #1	firmware Arduino/C++ con PubSubClient y SAS por mbedTLS
03	Python con SDK (`dc_sdk_mqtt.py`)	SDK oficial, MQTT/TLS 8883 con gemelo y métodos
04	Python con SDK sobre WebSockets (`dc_sdk_ws.py`)	mismo SDK, otro transporte : 443, atraviesa proxies
05	API pública meteorológica	el script consulta el feed (no inventa la variable)
06	API pública de calidad de aire	segundo feed , otro dominio, y otra ingesta: REST 443
07	ESP32 de Wokwi #2	segundo sketch, con sonda analógica y pulsador: no comparte código con el 02
08	Cliente paho explícito	sin SDK : SAS firmado a mano y registro DPS por MQTT
09	Replay de CSV histórico	no hay sensor: reproduce un histórico de la PDU con marca de tiempo de origen
10	Puente HTTP/REST + sensor	el sensor solo habla HTTP local (8098) y el puente traduce a MQTT

3. Banco de preguntas (con la respuesta corta)

¿Por qué el umbral de temperatura es 35 °C y el máximo dio 34,2 °C?

El datasheet del sensor de referencia (APC AP9335TH) da $\pm 0,5$ °C en el rango de la sala; el umbral de 35 °C está por debajo de la envolvente recomendada para el exhaust de un rack. El máximo de la ventana quedó en 34,2 °C justo por debajo: por eso la regla no disparó y se puede explicar como operación normal, no como fallo de la regla. Todas las variables tienen su tabla en el anexo A: rango del fabricante, rango operativo, precisión, valor usado en el código y umbral de la regla.

¿De dónde salen los valores de las variables que no tienen sensor real?

De modelos físicos simples documentados variable por variable en `docs/02-datasheets-y-parametros.md` (valor base, amplitud, offset y ruido). Las variables con fuente real (clima, calidad de aire y el histórico de la PDU) vienen de feeds públicos y del CSV, con la marca de tiempo de la fuente.

¿Qué transportes usa la flota y por qué?

MQTT/TLS en 8883 para la mayoría, MQTT sobre WebSockets en 443 para el nodo de pasillo (atraviesa proxies), HTTPS 443 para los feeds y el REST de dispositivo, y HTTP local en 8098 entre el sensor de campo y su puente. Todo el tráfico lo inician los dispositivos: el sitio es urbano con NAT saliente y ningún puerto de entrada abierto.

¿Qué hace DPS?

Asigna cada dispositivo a su hub con inscripción por clave simétrica (ID Scope + clave). En los sketches se ve el flujo completo: ``resp=202 status=assigning`` primero y ``resp=200 status=assigned`` después, sondeando con el ``operationId``.

¿Cuál es la desconexión que muestran?

Dos capas: una programada (el nodo de humo pausa su script en una ventana definida y deja hueco y estado Desconectado) y cuatro reales de la ventana (migración a servidor, doble publicación, SAS rechazado y cola de Wokwi). En vivo puedo reproducir la de ahora mismo bloqueando el dispositivo.

Limitaciones que encontramos en IoT Central (esta pregunta vale nota):

1. El simulador nativo no se puede parametrizar: da valores aleatorios 0–100, así que no sirve para comparativas térmicas serias y lo dejamos etiquetado aparte. 2. Las reglas y los mosaicos del panel no tienen API pública estable: la plantilla, los dispositivos y los grupos sí se automatizan, las reglas hay que hacerlas por la interfaz. 3. Las sesiones del portal caducan cada pocos minutos: obliga a trabajar por ráfagas. 4. Las cuotas de mensajes: con diez nodos y varios intervalos cortos hay que vigilar el consumo. 5. El formato de los mosaicos está poco documentado (hubo que reconstruirlo desde el OpenAPI). 6. El plan gratuito de Wokwi limita la cola de compilación cuando hay dos simulaciones a la vez.

¿Por qué el repositorio es público y cómo manejaron las credenciales?

Las claves viven en ``secrets/`` (ignorado) y se obtienen con ``tools/fetch_creds.py``. Antes de publicar corríamos una auditoría de tres capas —texto, OCR de todas las capturas y los PDF— y descubrimos que dos payloads de Wokwi tenían la clave incrustada: los sacamos del control de versiones y reescribimos el historial para que no quedaran en ningún commit.

¿Por qué el 25-sep y el 27-sep el nodo de agua no tiene datos?

El 25-sep la simulación del agua no estuvo publicando y el 27-sep los servidores de compilación de Wokwi no dejaron arrancarla (se verificó que no era el sketch, probando uno nuevo desde cero). Está declarado en el anexo: 9 de los 10 dispositivos tienen datos las cuatro fechas.

4. Mapa rúbrica → evidencia (para dirigir la conversación)

Indicador (peso)	Qué mostrar	Dónde
IoT Template · 15 %	plantilla publicada (42 capacidades), vistas Overview/About, propiedades escribibles, 4 comandos, logo e identidad del escenario	portal → Plantillas / Personalización; informe §5 y §9
Datos, Digital Twin y arquitectura · 20 %	catálogo de 10 filas, datasheets variable por variable, diagrama por capas con telecomunicaciones	`docs/01`, `docs/02`, `evidencias/diagrama_arquitectura.png`, informe §3–§4
Heterogeneidad de orígenes · 20 %	tabla de trazabilidad (por qué cada origen es distinto), seis intervalos, la desconexión en vivo y los cuatro incidentes	`docs/01` (trazabilidad), anexo 4.3, informe §6.1
Ventana de 4 días · 15 %	consulta del Data Explorer 24–27 sep + la comparativa (máx/mín/promedio/recuento/sumatoria) y la lectura operativa	portal → Explorador de datos; `docs/04`; anexo 4.4; informe anexo B
Control room y documento · 15 %	panel con logo, KPIs, 8 gráficos y alertas; historial de versiones; repo limpio y público	panel del portal; informe (portada y versiones); repositorio en GitHub
Sustentación y dos códigos en vivo · 15 %	los dos equipos publicando, la desconexión y reconexión, y las respuestas técnicas	esta sección 1 del guion

5. Plan de contingencia

Si falla...	Qué hacer
La red del sitio	mostrar las capturas de las cuatro fechas y los registros ya guardados; explicar que los orígenes siguen configurados
El servidor Ubuntu (está apagado)	usar el **portátil** como equipo 1 (es exactamente el mismo código Python) y decir que el servidor se usó en las fechas 3 y 4
Wokwi (cola de compilación)	mostrar la evidencia histórica del monitor serie de las fechas 3 y 4 y usar como segundo equipo el **puente de API pública** (`dc_api_openmeteo.py`), que corre con peticiones HTTPS normales
El portal no deja iniciar sesión	abrir sesión primero en el navegador que ya la tiene guardada y entrar directo a la URL de la app
Una regla no dispara	es normal: los umbrales están por encima de los máximos de la ventana; explicar la lectura operativa de cada extremo

6. Frase de cierre

"La flota no se llenó con un solo simulador: diez dispositivos, diez orígenes, seis intervalos, una desconexión documentada y cuatro incidentes reales resueltos durante la ventana. El cuarto de control responde de un vistazo si el ambiente de los racks está en envolvente, si conviene enfriar con aire exterior y si hay eventos de agua, humo o acceso. Todo lo que afirmo está en el repositorio público o en el informe."